

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

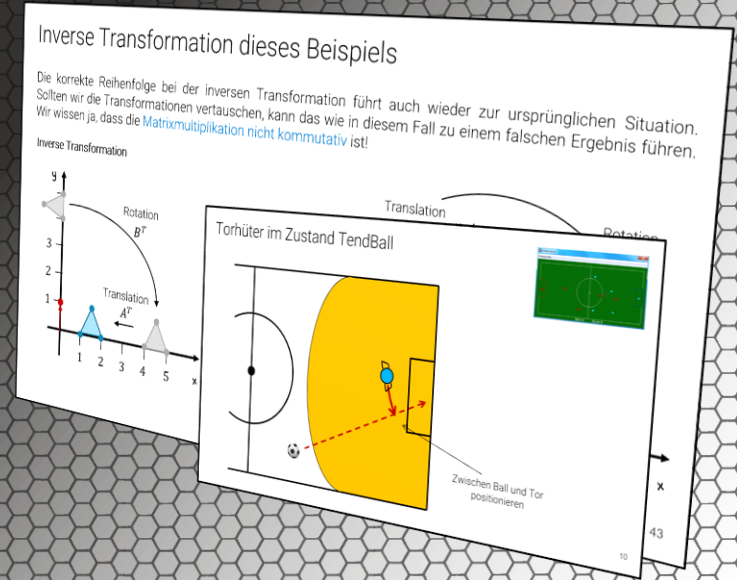
Sommersemester 2026



Transformationen als Vollzeitjob

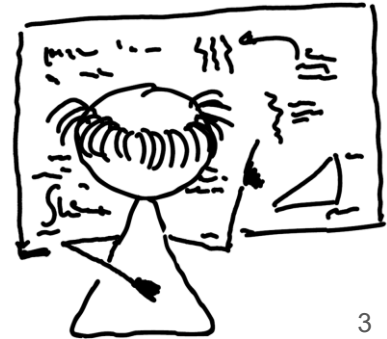
„Nichts ist so beständig wie der Wandel.“ (Heraklit von Ephesus)

PD Prof. Dr. Marco Block-Berlitz



Ziele und Inhalt

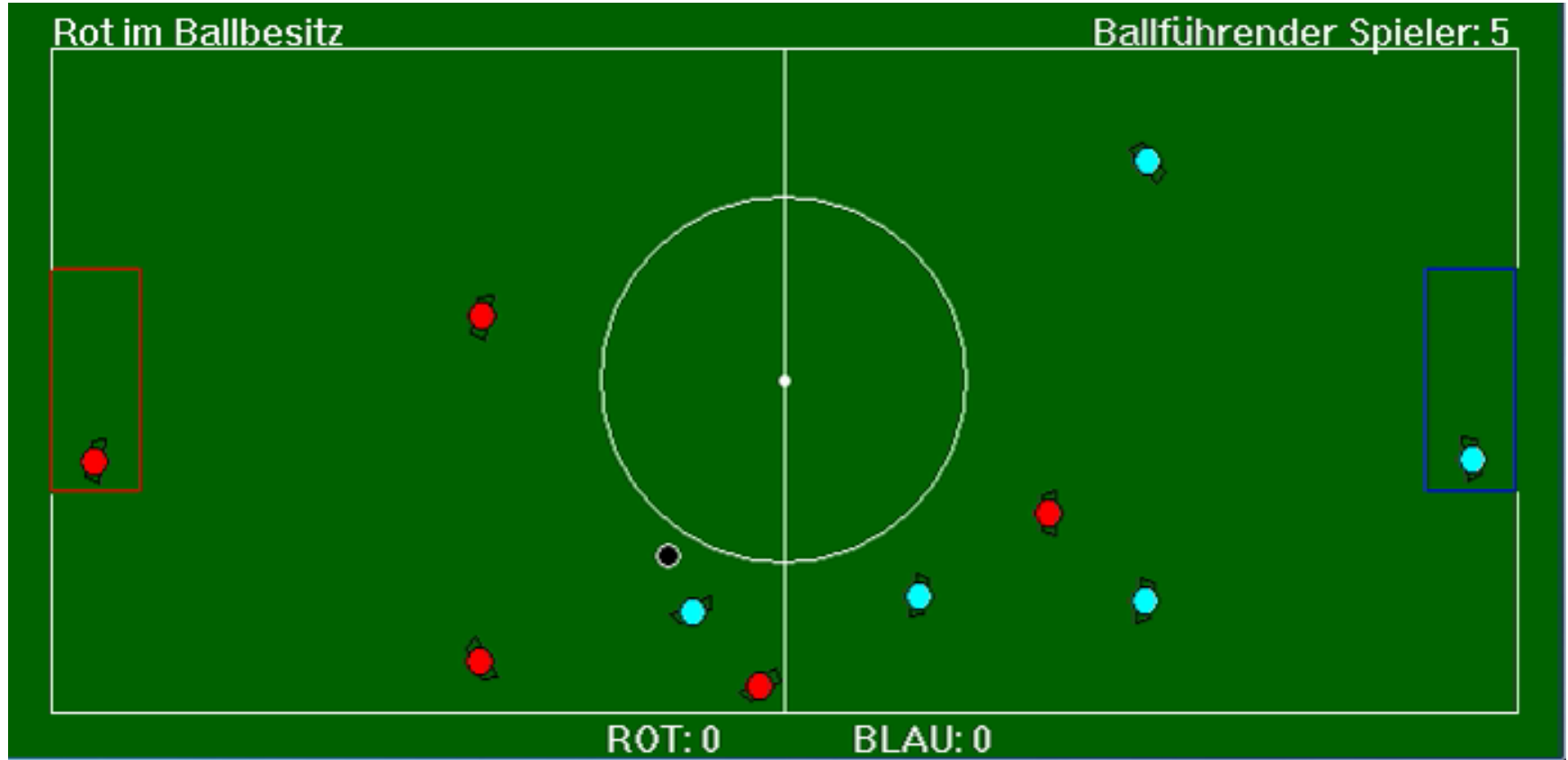
- Beispiel SimpleSoccer
- Schlüsselmethode: Sicherheit eines Passes
- Motivation zu Koordinatentransformationen
- Vektor von A nach B transformieren
- Vektor von B nach A transformieren
- Konkretes Rotationsbeispiel in 2D
- Hin- und Rücktransformation
- Beispiel mit EJML
- Herleitung der Rotationsmatrix
- Rotation und Translation
- Homogene Koordinaten
- Translation mit homogenen Koordinaten
- Skalierung mit homogenen Koordinaten
- Rotationen mit homogenen Koordinaten
- Transformationen beliebig kombinieren
- Beispiel: Bewegendes Objekt
- Performance-Vergleich – Vorbereitung
- Performance-Vergleich – Ergebnis
- Vergleich der Ergebnisse – Konsequenz
- Inverse Transformation dieses Beispiels
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



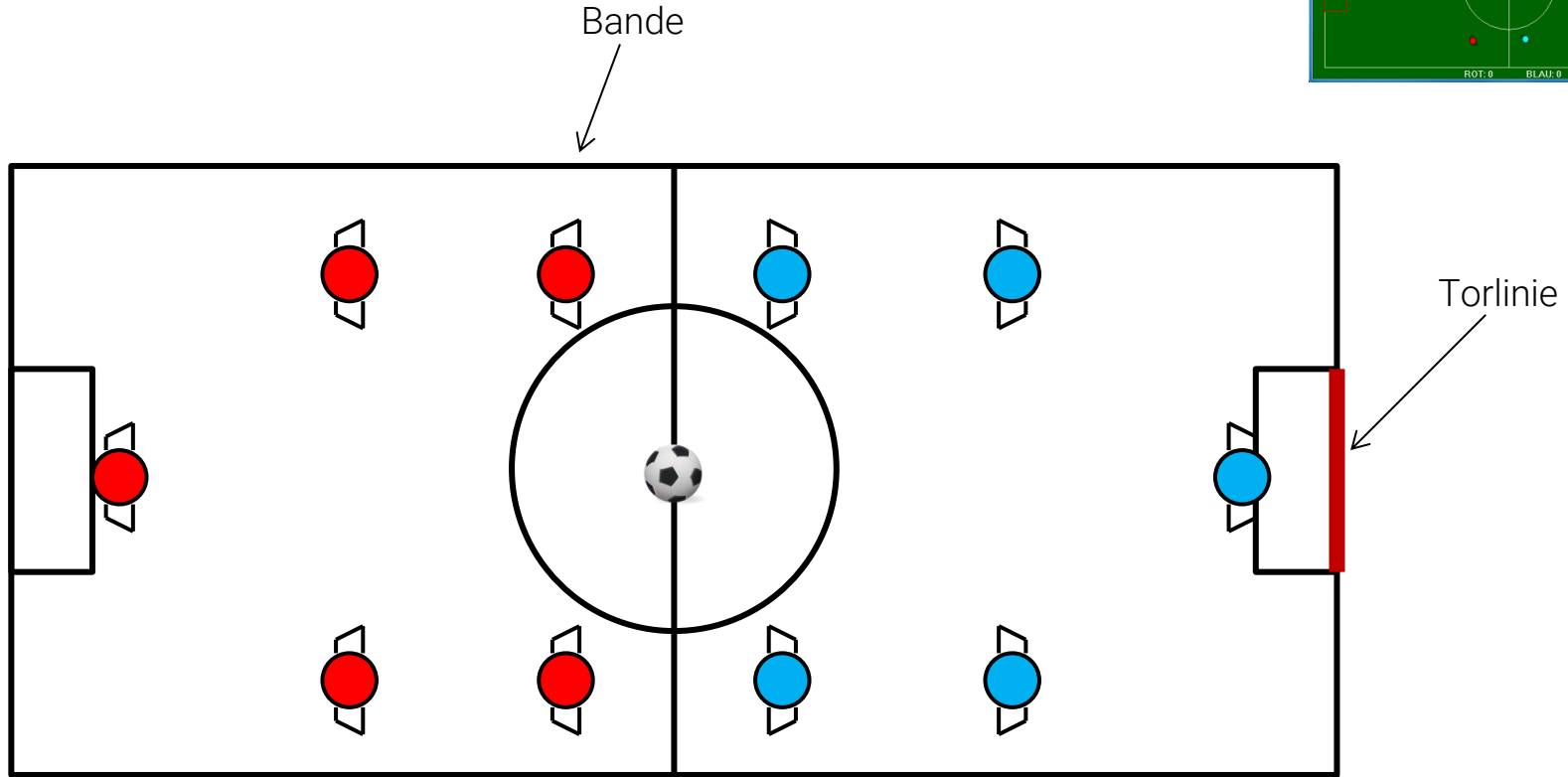
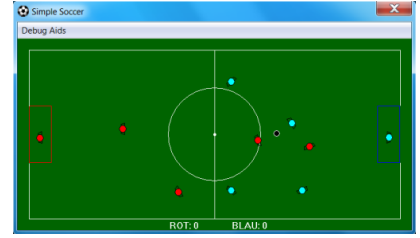


Bevor wir loslegen, wollen wir uns kurz für
Koordinatentransformationen motivieren ...

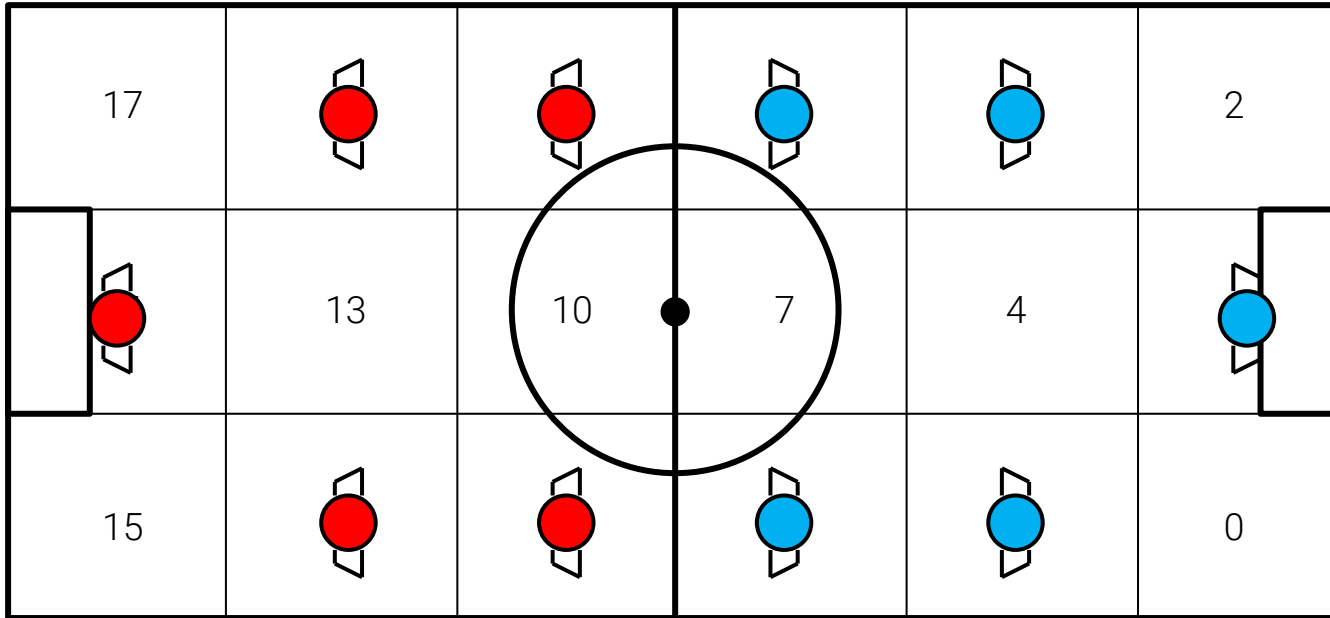
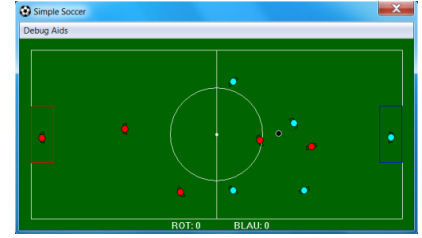
SimpleSoccer – Eine einfache Fußballsimulation [1]



Hallenfußball 5 gegen 5

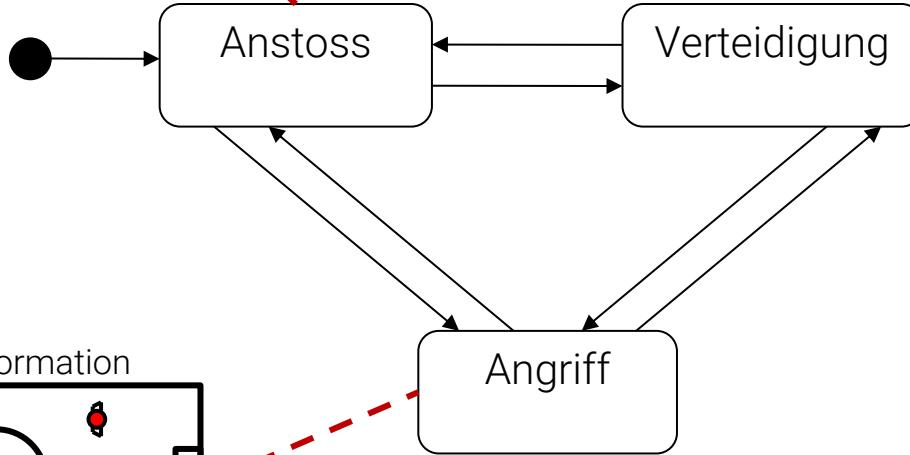
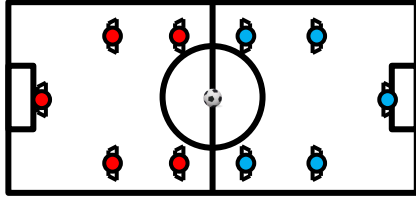


Feldaufteilung für Positionierung

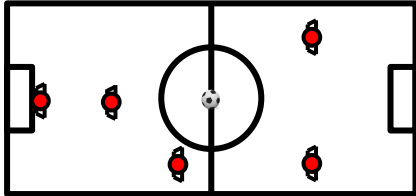


Drei Zustände der Mannschaften

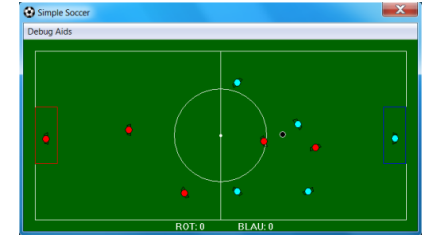
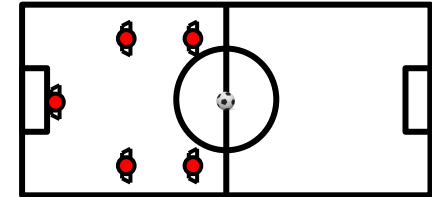
Startpositionen der Spieler



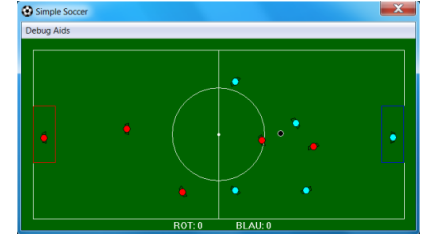
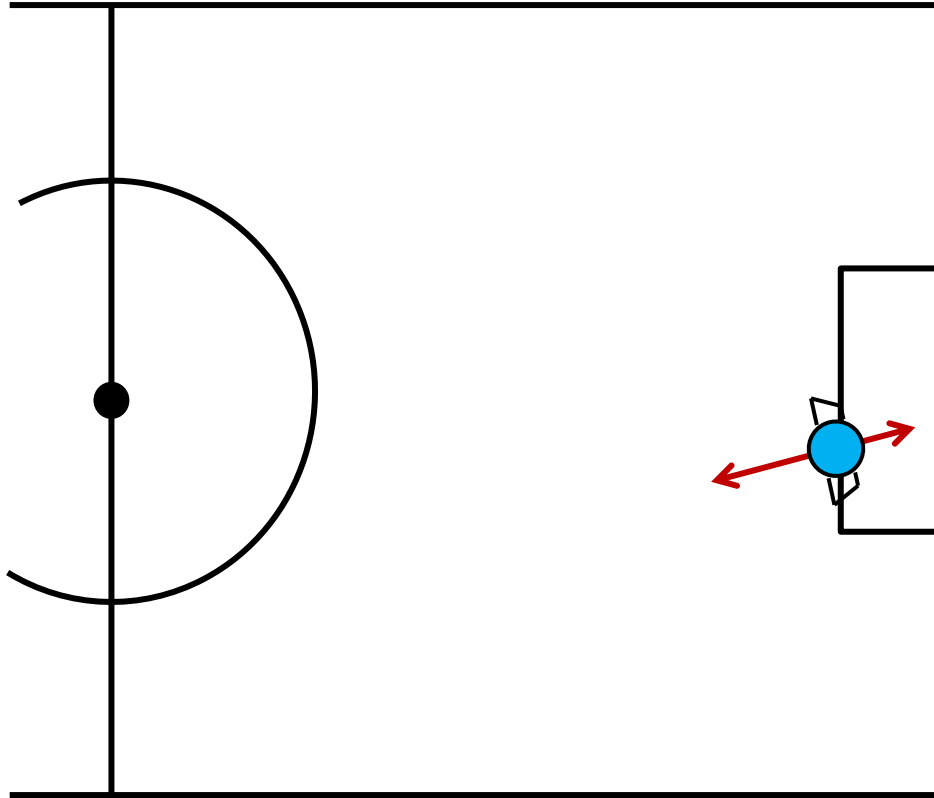
Angriffsformation



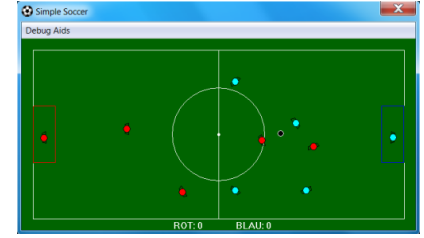
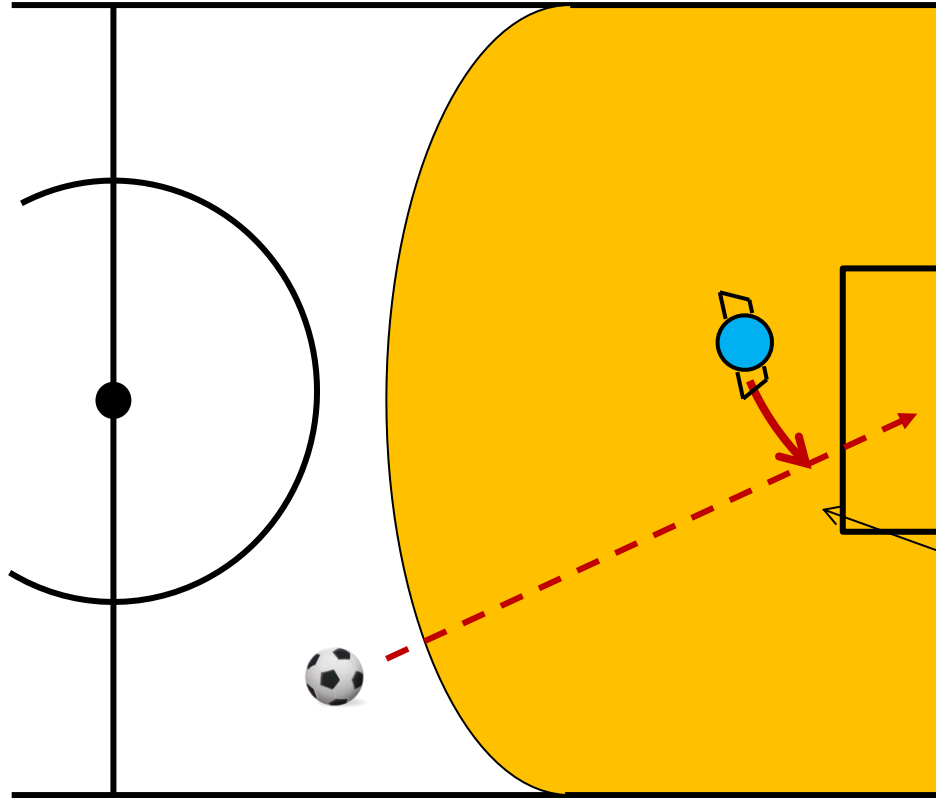
Verteidigungsformation



Bewegungen des Torhüters

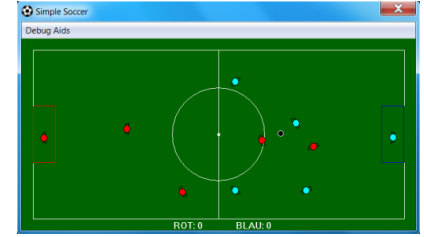
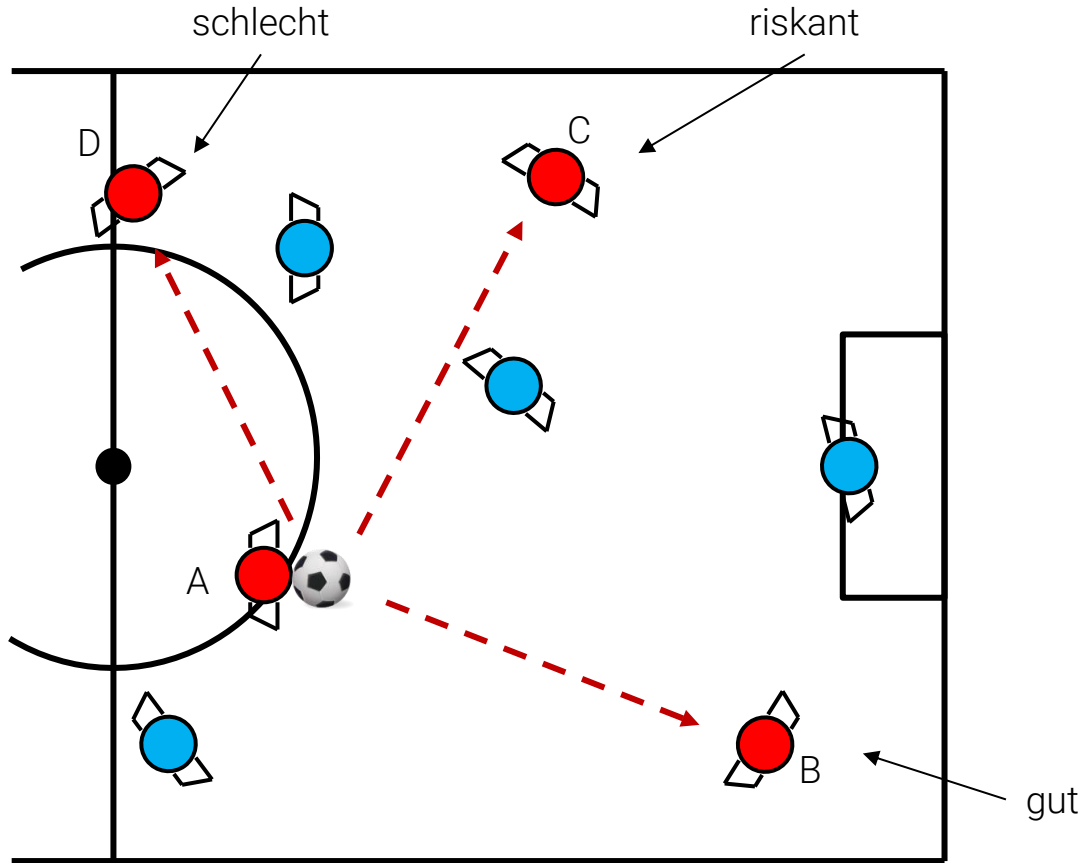


Torhüter im Zustand TendBall

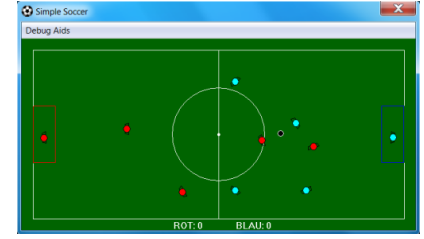
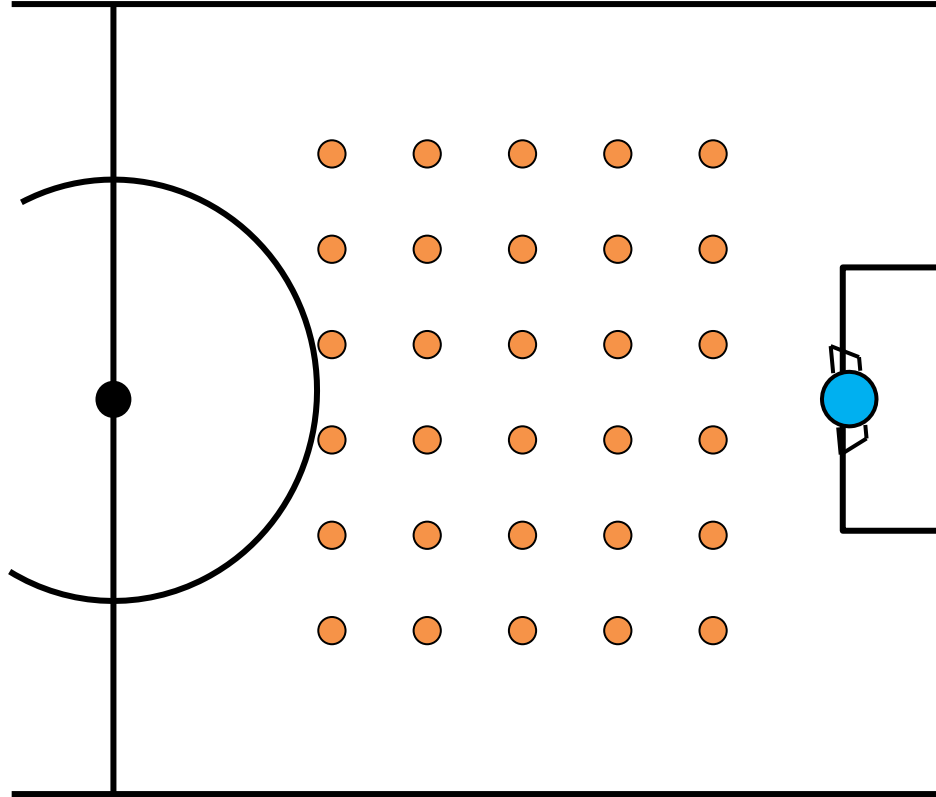


Zwischen Ball und Tor
positionieren

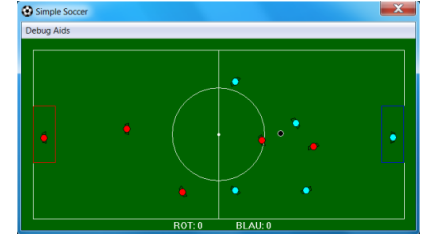
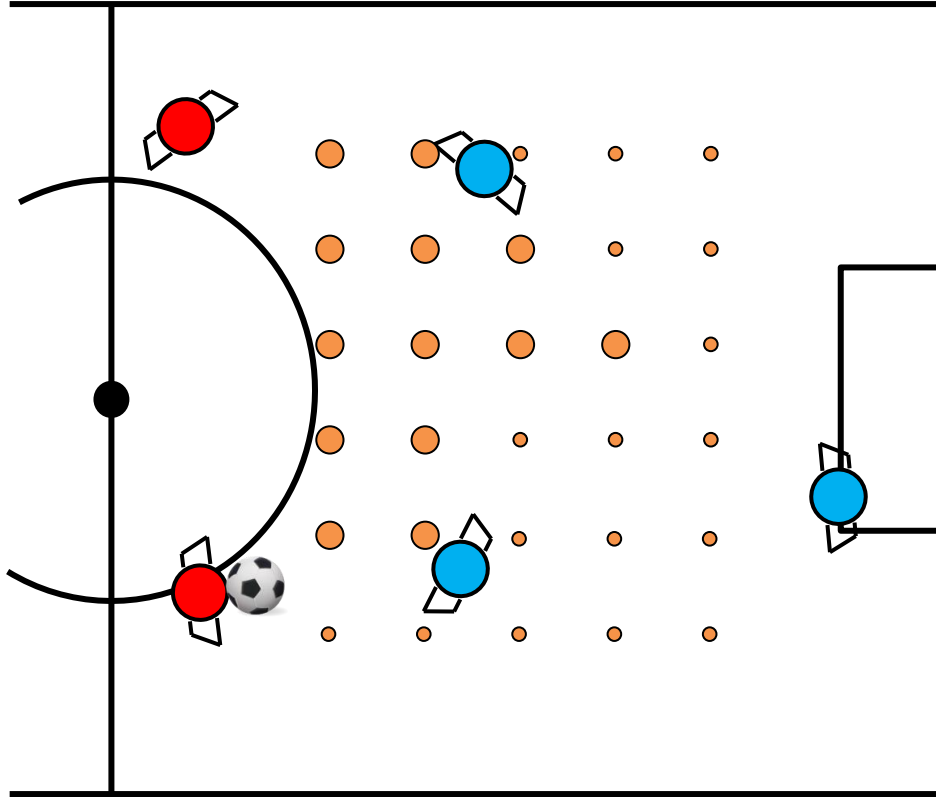
Grobe Bewertung der Mitspieler



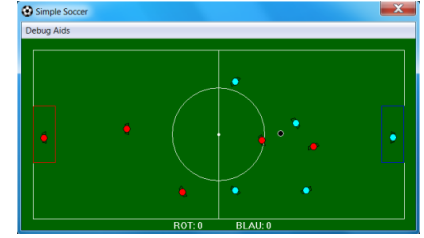
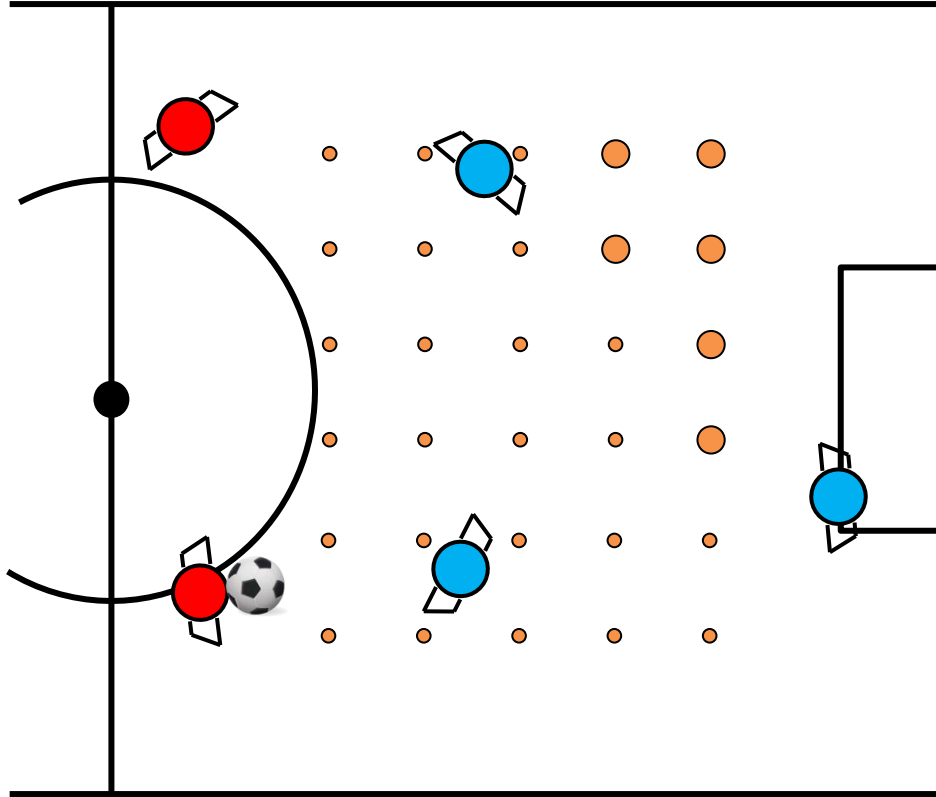
Ermittlung des Best Support Spot (BSS)



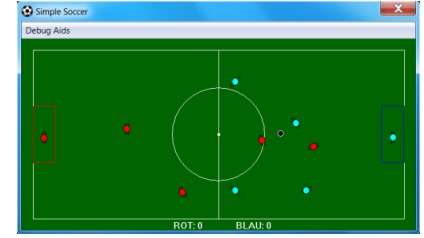
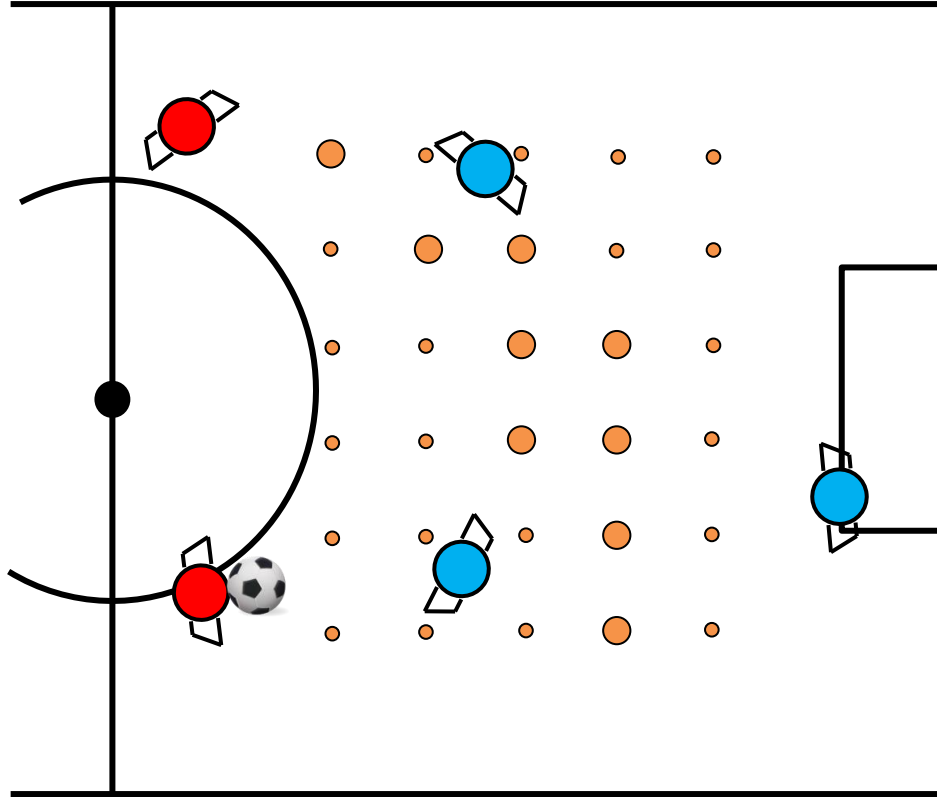
BSS – Potential für weitere Pässe



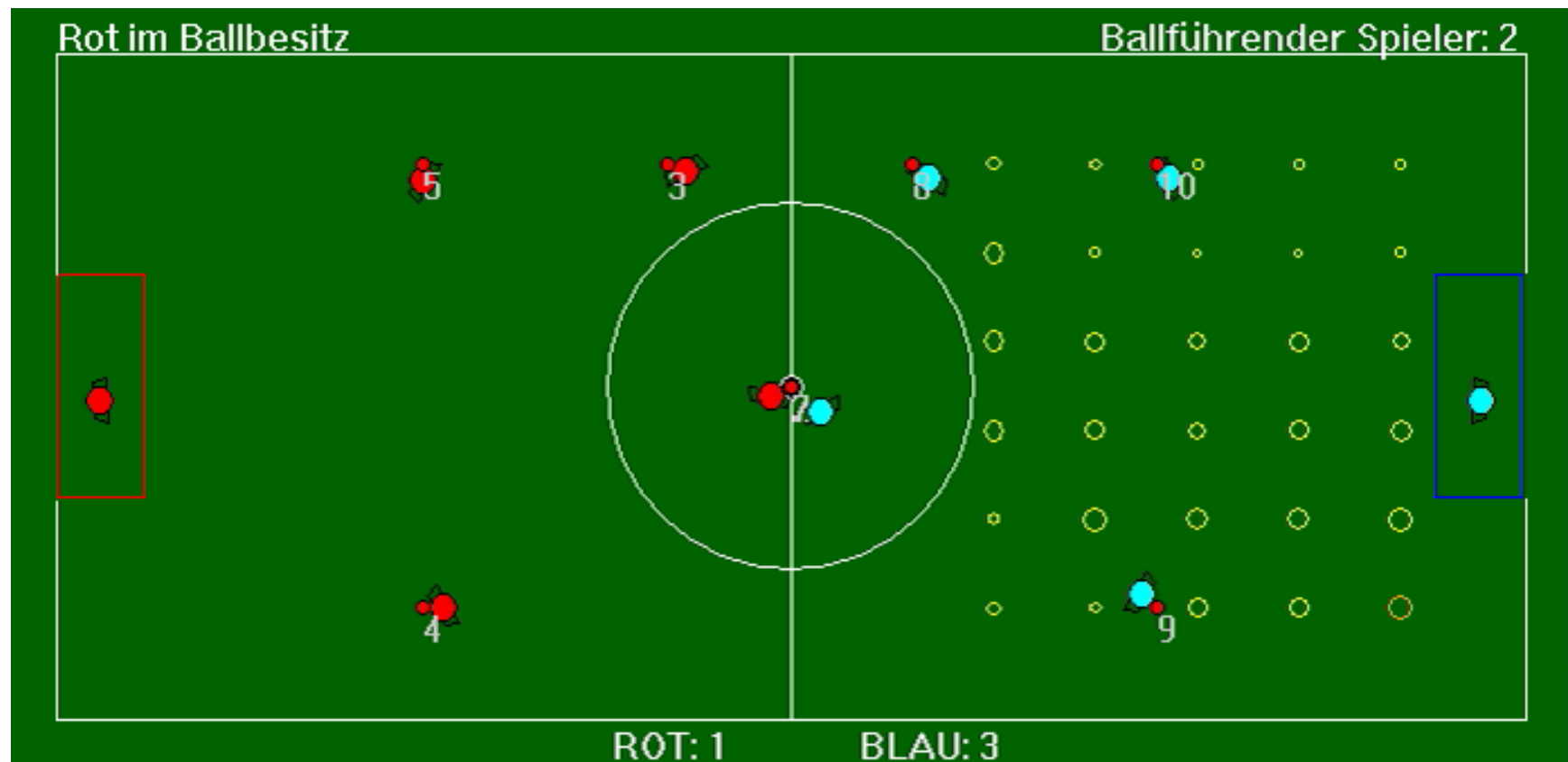
BSS – Potential für Torschuss



BSS – Entfernung zu Gegenspieler



SimpleSoccer – Visualisierung des BSS

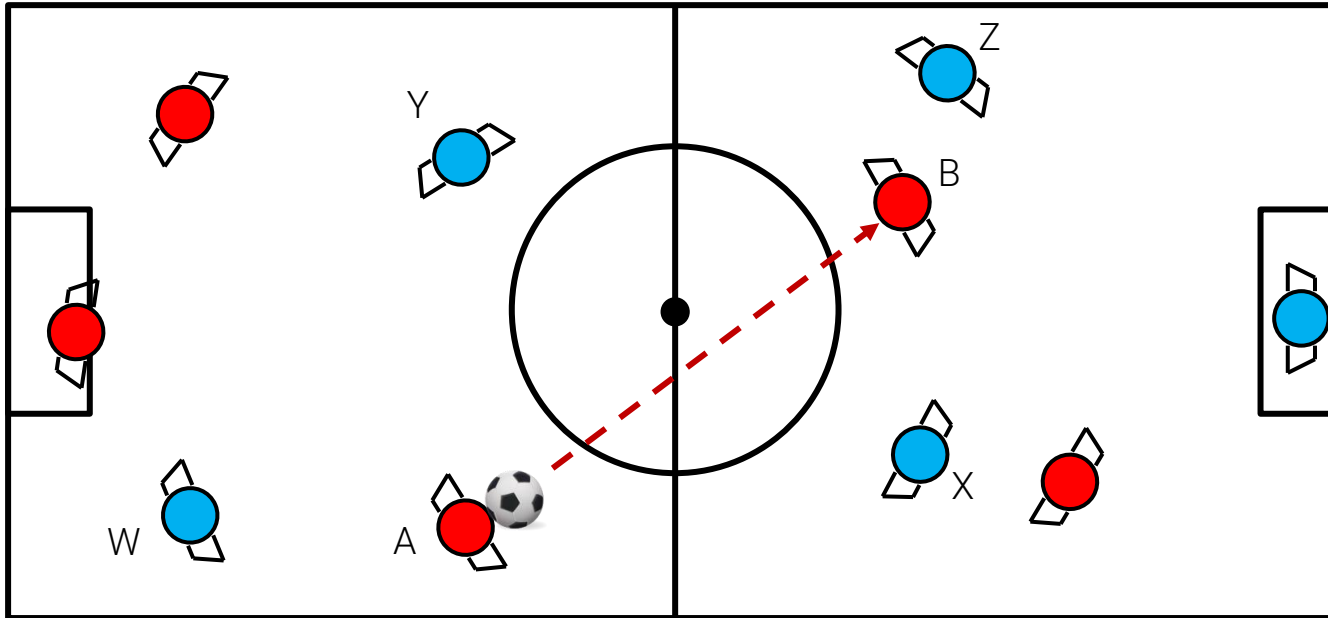
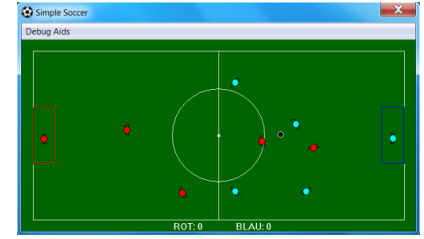




Jetzt kennen wir den Kontext und kommen
zu einer konkreten Aufgabe ...

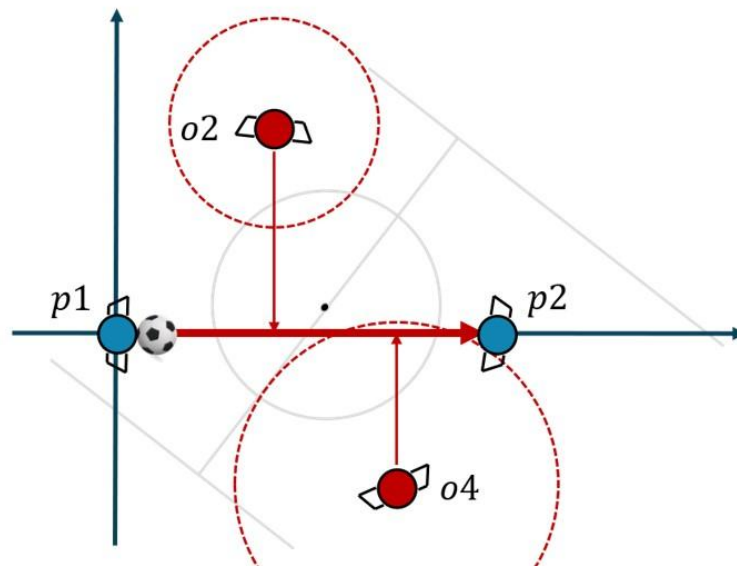
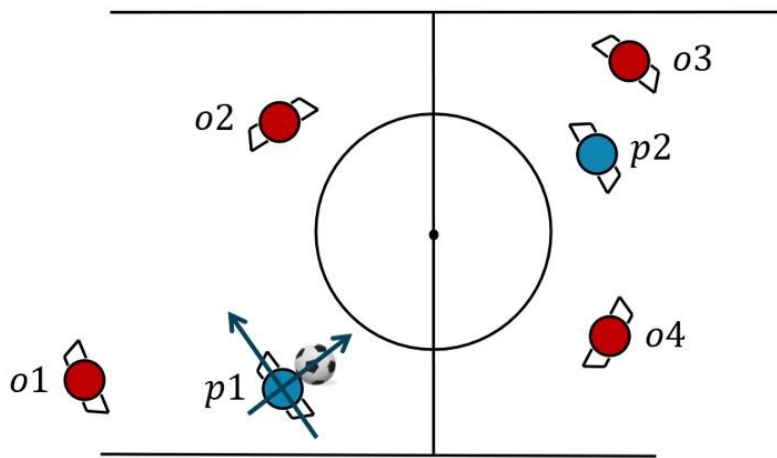
Schlüsselmethode: Sicherheit eines Passes

Wir haben ein [globales Feldkoordinatensystem](#). Die Entscheidung, ob Spieler *A* einen sicheren Pass zu Spieler *B* spielen kann lässt sich deutlich einfacher im [lokalen Koordinatensystem](#) des Spielers *A* entscheiden.



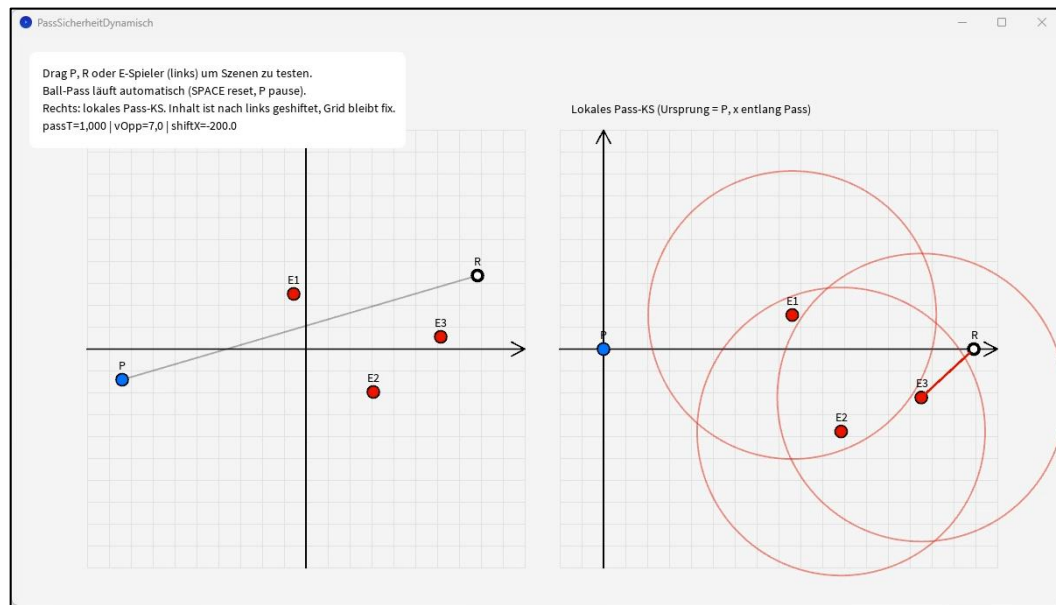
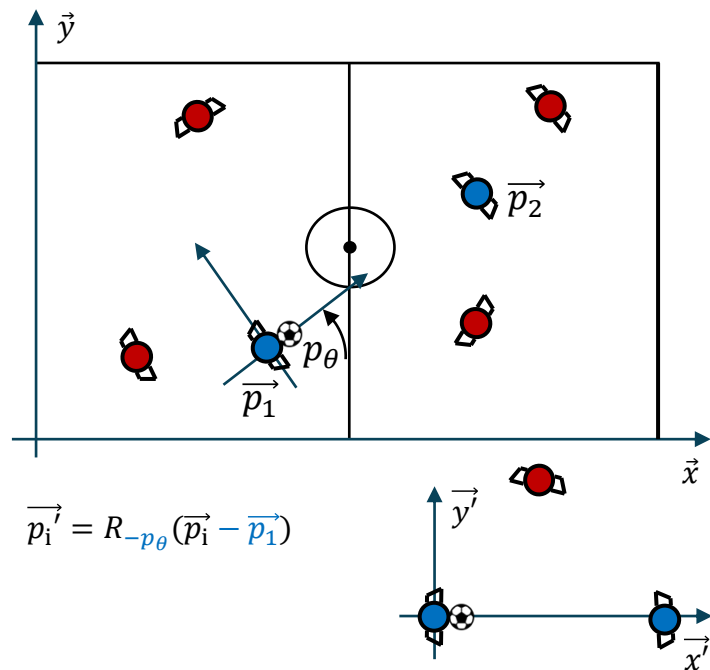
Motivation zu Koordinatentransformationen

Eine Schlüsselsituation aus der Fußballsimulation SimpleSoccer [1]. Es soll entschieden werden, ob ein Pass von Spieler p_1 zu Spieler p_2 sicher ist, also nicht von den umliegenden blauen Gegenspielern abgefangen werden kann. Die Entscheidung fällt leichter, wenn wir alle dafür relevanten Dinge in ein [Koordinatensystem](#) relativ zum Spieler p_1 [transformieren](#). Jetzt können wir den direkten Abstand des Gegenspielers zur Laufrichtung des Balles bestimmen und die Entfernung berücksichtigen, die ein Spieler zurücklegen kann, solange der Ball unterwegs ist.



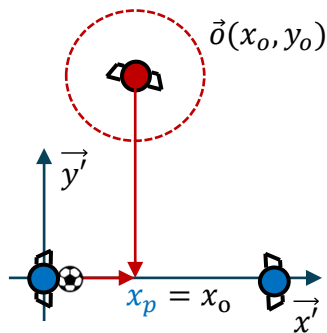
Koordinatentransformation zu Spieler

Schauen wir uns an, wie das in einem dynamischen Setup aussehen könnte. Dazu können wir die Spieler auf dem Feld auch verschieben. Die erste Aufgabe lautet, alle wichtigen Spielelemente aus den **Weltkoordinaten in das lokale Koordinatensystem des Spielers** zu bringen, wobei die x -Achse der Passrichtung entspricht.



Entscheidung über die Passsicherheit

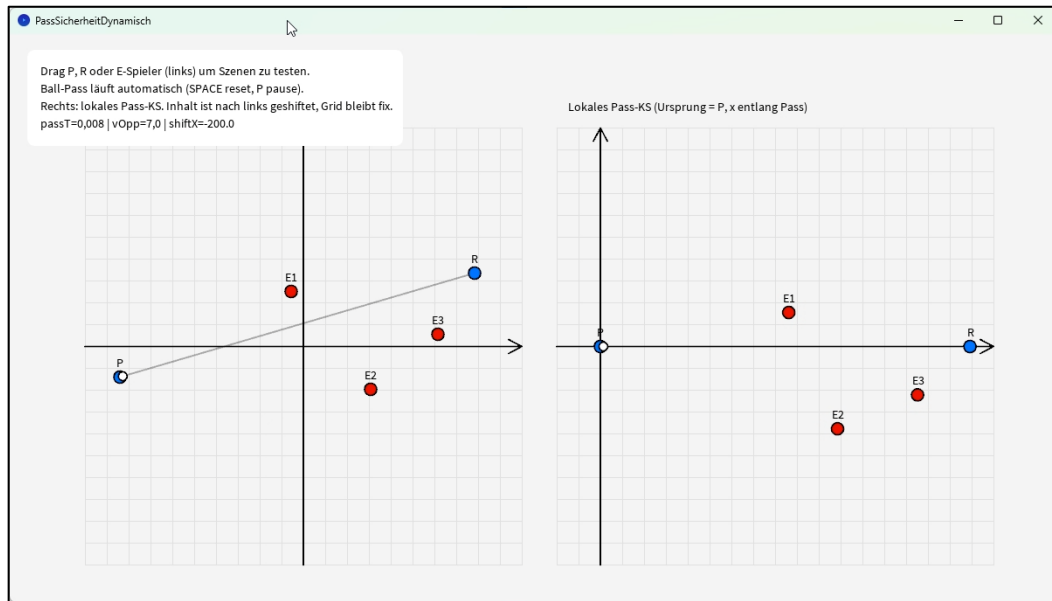
Entscheidung, ob der Pass zum Empfänger $\vec{p}_r = (x_r, 0)$ sicher ist oder nicht, mit $x_r > 0$. Ball bewegt sich mit konstanter Geschwindigkeit v_b . Der relevante Abfangpunkt auf der Passlinie ist $x_p = x_o$. Zeit bis zu der Position x_p , die der Gegner am schnellsten erreichen kann, ist $t_b = x_p/v_b$. Ein Gegner (x_o, y_o) kann sich mit max. v_o bewegen. Zeit, die der Gegner braucht, um Position x zu erreichen (da $0 \leq x_o \leq x_r$) ist $t_o = |y_o|/v_o$.



Ein Pass ist **sicher**, wenn $t_o > t_b$ bzw.

$$\frac{|y_o|}{v_o} > \frac{x_p}{v_b}$$

entsprechend **unsicher**, sonst.



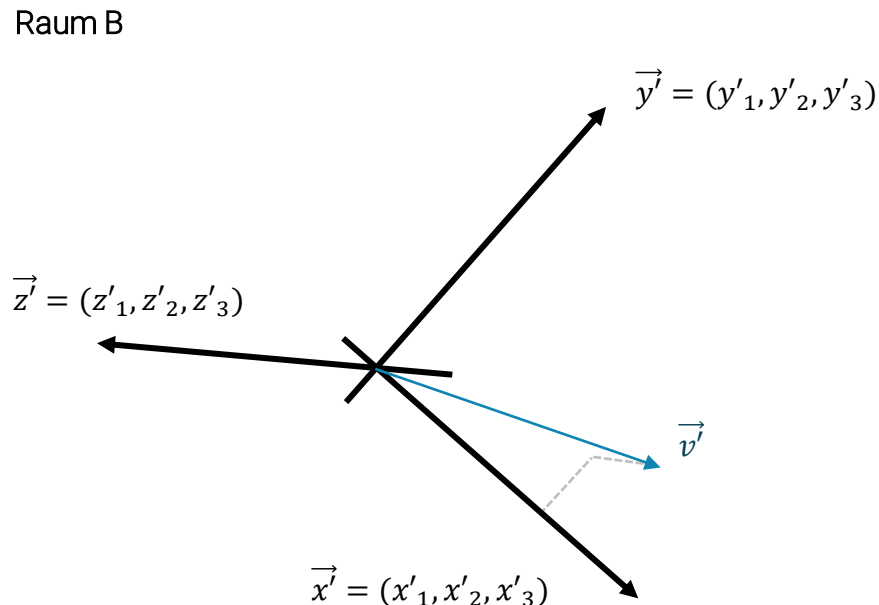
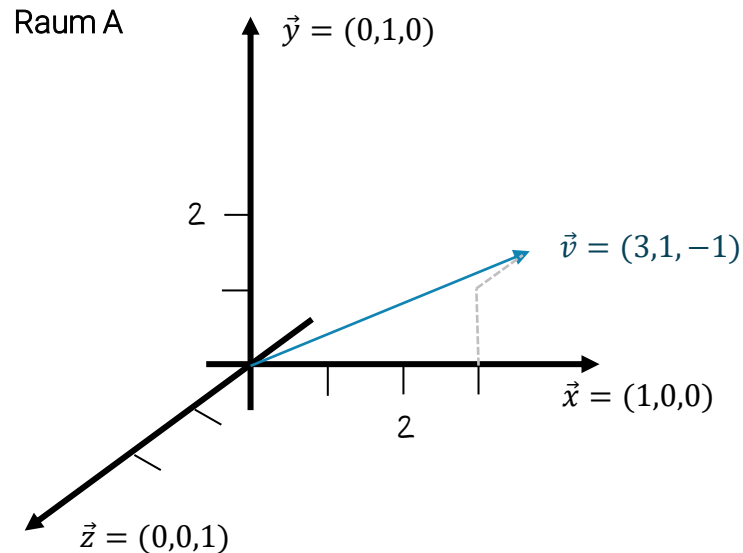


Wir brauchen **Koordinatentransformationen**, weil Entscheidungen einfacher werden, wenn wir ein geeignetes Bezugssystem nutzen.

Beispielvektor von A nach B transformieren

Angenommen, es liegen zwei orthogonale Vektorräume A und B vor. Wir können mit einer einfachen Vorschrift einen Vektor $\vec{v}$ aus A in B transformieren mit:

$$\vec{v}' = B\vec{v} = \vec{v}' = \begin{bmatrix} \vec{x}' & \vec{y}' & \vec{z}' \end{bmatrix} \vec{v} = \begin{bmatrix} x'_1 & y'_1 & z'_1 \\ x'_2 & y'_2 & z'_2 \\ x'_3 & y'_3 & z'_3 \end{bmatrix} \vec{v}$$



Beispielvektor wieder von B nach A transformieren

Jetzt wollen wir den neuen Vektor $\vec{v'}$ wieder zurücktransformieren und schauen, ob wir mit der **inversen Matrix** von B wieder in A den ursprünglichen Vektor $\vec{v}$ erhalten:

$$\vec{v} = B^{-1}\vec{v'}$$

Hier nutzen wir die Annehmlichkeit „orthogonaler Räume“:

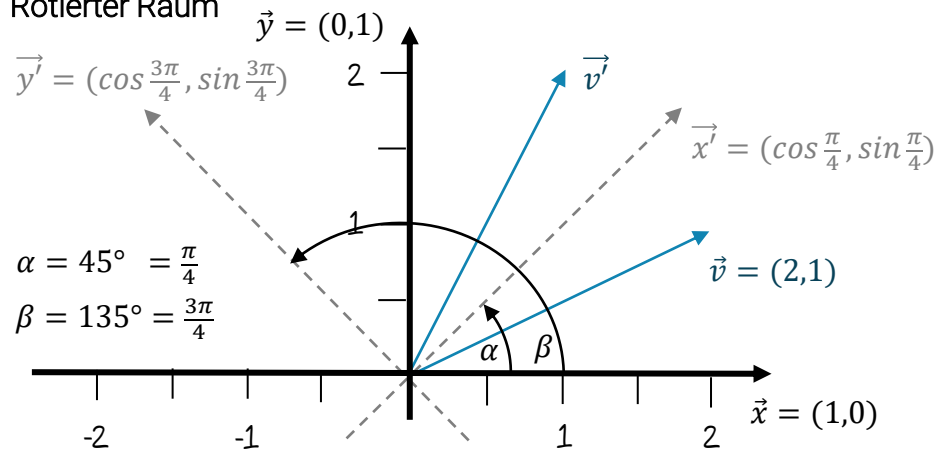
$$\vec{v} = B^{-1}\vec{v'} = \begin{bmatrix} \vec{x'} & \vec{y'} & \vec{z'} \end{bmatrix}^{-1} \vec{v'} = \begin{bmatrix} \vec{x'}^T \\ \vec{y'}^T \\ \vec{z'}^T \end{bmatrix} \vec{v'} = \begin{bmatrix} x'_1 & x'_2 & x'_3 \\ y'_1 & y'_2 & y'_3 \\ z'_1 & z'_2 & z'_3 \end{bmatrix} \vec{v'}$$



Konkretes Rotationsbeispiel in 2D

Ein Beispiel dazu aus dem 2-dimensionalen Raum. Wir wollen Vektoren aus dem Koordinatensystem A in das um 45° gedrehte System B transformieren:

Rotierter Raum

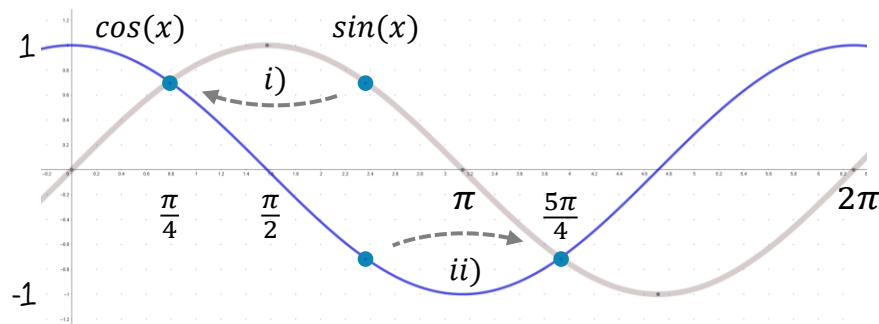


Zwei nützliche Umformungen ergeben sich hier:

$$\cos\left(\frac{3\pi}{4}\right) = \sin\left(\frac{\pi}{2} - \frac{3\pi}{4}\right) = \sin\left(\frac{2\pi}{2} - \frac{3\pi}{4}\right) = \sin\left(-\frac{\pi}{4}\right) = -\sin\left(\frac{\pi}{4}\right)$$

$$\sin\left(\frac{3\pi}{4}\right) = -\sin\left(-\frac{3\pi}{4}\right) = \cos\left(-\frac{3\pi}{4} + \frac{\pi}{2}\right) = \cos\left(-\frac{3\pi}{4} + \frac{2\pi}{4}\right) = \cos\left(-\frac{\pi}{4}\right) = \cos\left(\frac{\pi}{4}\right)$$

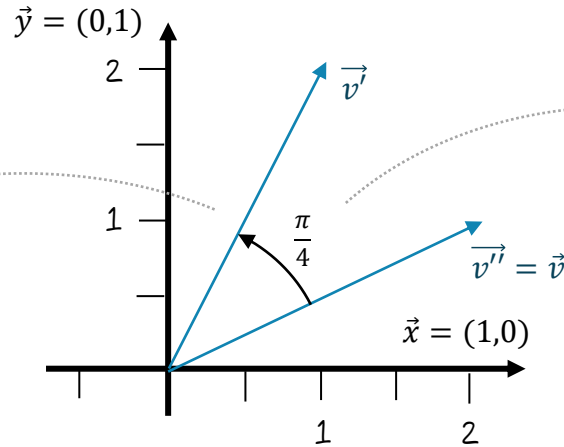
Funktionsverläufe $\sin(x)$ und $\cos(x)$



Hin- und Rücktransformation

Transformieren wir den Vektor $\vec{v} = (2,1)$ von A nach B komplett (links) und zeigen anschließend mit Hilfe der transponierten Matrix, dass der ursprüngliche Vektor $\vec{v}$ erfolgreich reproduziert werden kann:

$$\begin{aligned}
 \vec{v}' &= B\vec{v} \\
 &= [\vec{x}' \quad \vec{y}'] \cdot \vec{v} \\
 &= \begin{bmatrix} \cos \frac{\pi}{4} & \cos \frac{3\pi}{4} \\ \sin \frac{\pi}{4} & \sin \frac{3\pi}{4} \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \frac{\pi}{4} & -\sin \frac{\pi}{4} \\ \sin \frac{\pi}{4} & \cos \frac{\pi}{4} \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} 2 \cdot \cos \frac{\pi}{4} - \sin \frac{\pi}{4} \\ 2 \cdot \sin \frac{\pi}{4} + \cos \frac{\pi}{4} \end{bmatrix} \\
 &\approx \begin{bmatrix} 0.7 \\ 2.1 \end{bmatrix}
 \end{aligned}$$



$$\begin{aligned}
 \vec{v}'' &= B^T \vec{v}' \\
 &= [\vec{x}' \quad \vec{y}']^T \cdot \vec{v}' \\
 &= \begin{bmatrix} \cos \frac{\pi}{4} & \sin \frac{\pi}{4} \\ -\sin \frac{\pi}{4} & \cos \frac{\pi}{4} \end{bmatrix} \cdot \begin{bmatrix} 2 \cdot \cos \frac{\pi}{4} - \sin \frac{\pi}{4} \\ 2 \cdot \sin \frac{\pi}{4} + \cos \frac{\pi}{4} \end{bmatrix} \\
 &= \begin{bmatrix} \cos \frac{\pi}{4} \cdot (2 \cdot \cos \frac{\pi}{4} - \sin \frac{\pi}{4}) + \sin \frac{\pi}{4} \cdot (2 \cdot \sin \frac{\pi}{4} + \cos \frac{\pi}{4}) \\ -\sin \frac{\pi}{4} \cdot (2 \cdot \cos \frac{\pi}{4} - \sin \frac{\pi}{4}) + \cos \frac{\pi}{4} \cdot (2 \cdot \sin \frac{\pi}{4} + \cos \frac{\pi}{4}) \end{bmatrix} \\
 &= \begin{bmatrix} 2 \cdot \cos^2 \frac{\pi}{4} - \cos \frac{\pi}{4} \cdot \sin \frac{\pi}{4} + 2 \cdot \sin^2 \frac{\pi}{4} + \cos \frac{\pi}{4} \cdot \sin \frac{\pi}{4} \\ -\sin \frac{\pi}{4} \cdot 2 \cdot \cos \frac{\pi}{4} + \sin^2 \frac{\pi}{4} + \cos \frac{\pi}{4} \cdot 2 \cdot \sin \frac{\pi}{4} + \cos^2 \frac{\pi}{4} \end{bmatrix} \\
 &= \begin{bmatrix} 2 \cdot \cos^2 \frac{\pi}{4} + 2 \cdot \sin^2 \frac{\pi}{4} \\ \sin^2 \frac{\pi}{4} + \cos^2 \frac{\pi}{4} \end{bmatrix} \\
 &= \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\
 &= \vec{v}
 \end{aligned}$$

Wir erhalten nach der Rücktransformation wie erwartet $\vec{v}'' = \vec{v}$.

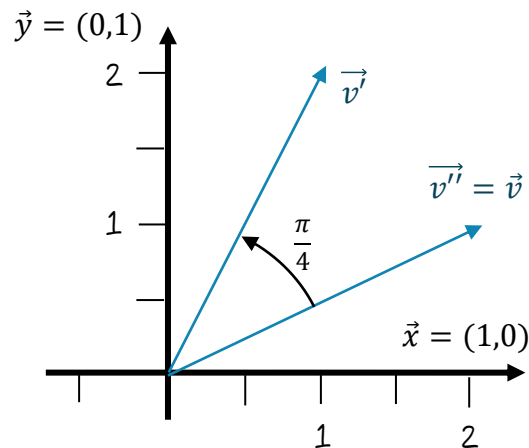
Beispiel mit EJML

Schauen wir uns das Beispiel in EJML an:

```
SimpleMatrix R = new SimpleMatrix(new double[][] {  
    {Math.cos(Math.PI/4), -Math.sin(Math.PI/4)},  
    {Math.sin(Math.PI/4), Math.cos(Math.PI/4)}});  
SimpleMatrix V = new SimpleMatrix(new double[][] {{2},{1}});  
SimpleMatrix V2 = R.mult(V);  
System.out.println(V2);  
  
SimpleMatrix R2 = R.transpose();  
System.out.println(R2.mult(V2));
```

Wir erhalten:

```
Type = DDRM , rows = 2 , cols = 1  
0,707  
2,121  
  
Type = DDRM , rows = 2 , cols = 1  
2,00  
1,00
```



Herleitung der Rotationsmatrix

Zusammenfassend können wir nun folgendes angeben:

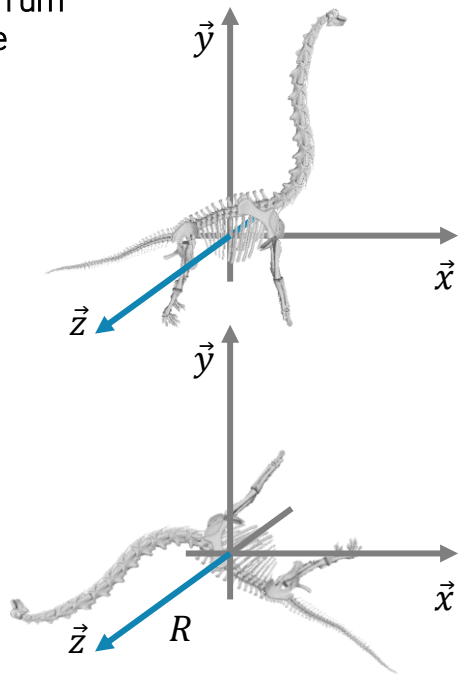
$$R = [\vec{x'} \quad \vec{y'}]} = \begin{bmatrix} \cos \frac{\pi}{4} & \cos \frac{3\pi}{4} \\ \sin \frac{\pi}{4} & \sin \frac{3\pi}{4} \end{bmatrix} = \begin{bmatrix} \cos \frac{\pi}{4} & -\sin \frac{\pi}{4} \\ \sin \frac{\pi}{4} & \cos \frac{\pi}{4} \end{bmatrix}$$

Da wir aber um den speziellen Winkel $\alpha = \frac{\pi}{4}$ rotiert haben, können wir die allgemeine **Rotationsmatrix** direkt ableiten:

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Wir können damit zwar die Achsen in verschiedene Richtungen rotieren, aber der **Ursprung** bleibt jeweils der gleiche.

Rotation um
z-Achse





Bisher können wir nur um den **Ursprung** rotieren,
was brauchen wir für die Rotation um einen
beliebigen Punkt?

Rotation und Translation

Wenn wir **Rotation und Translation** verbinden wollen, benötigen wir also noch einen Translationsvektor $\vec{t}$:

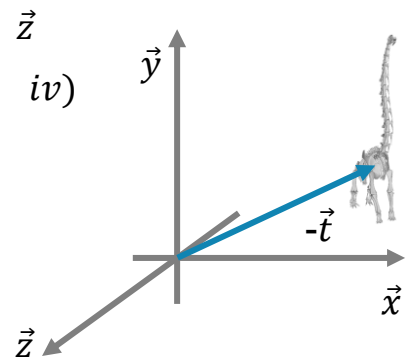
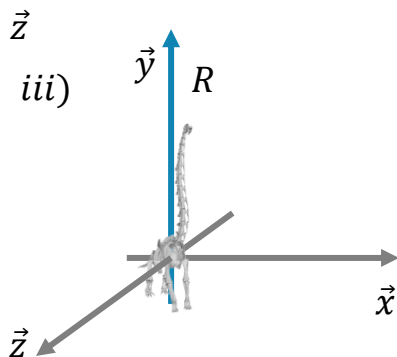
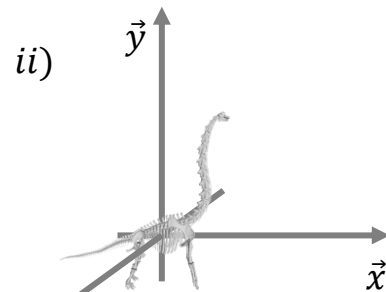
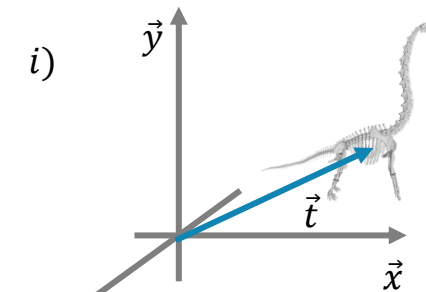
$$\vec{v}' = R_{\theta} \vec{v} + \vec{t}$$

Das führt uns zu:

$$\vec{v}' = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Um ein Objekt um eine beliebige Achse zu drehen, müssen wir es zunächst zum Ursprung bringen.

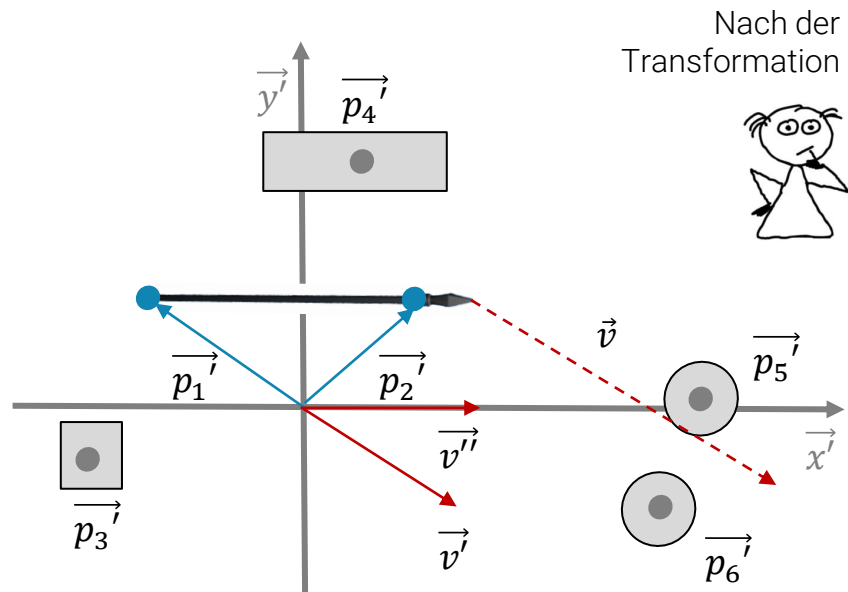
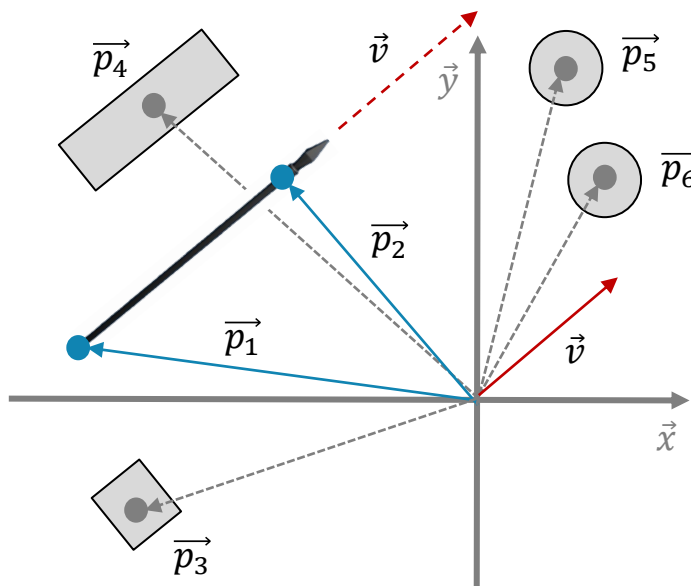
Translation und Rotation um y-Achse



Homogene Koordinaten

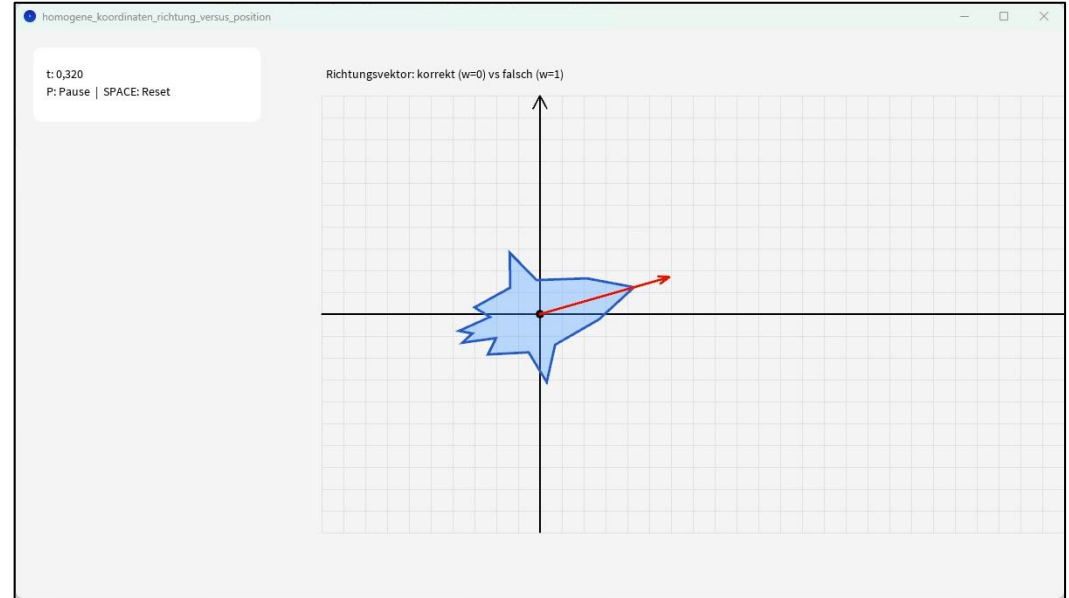
Zwei Dinge machen die Form der Transformation aus dem vorhergehenden Teil nicht besonders schön:

- Eine Transformation benötigt **zwei Verarbeitungsstufen** und
- die Transformation **unterscheidet nicht** zwischen Vektoren, die Punkte repräsentieren, und Vektoren, die Richtungen repräsentieren.



Schauen wir uns ein Beispiel an

Hier sehen wir nochmal ein dynamisches Beispiel. Ein Raumschiff besteht aus vielen Positionsvektoren und einem Richtungsvektor. Wenn sich das Raumschiff im Ursprung rotiert, ist die Problematik noch nicht ersichtlich und alles wirkt korrekt. Erst wenn sich das Raumschiff in Bewegung versetzt (Translation), dann darf der Richtungsvektor diese Transformation nicht mitmachen (gestrichelte rote Linie).



Homogene Koordinaten

Wir erweitern die 2D-Vektoren zu Vektoren in **homogenen Koordinaten** und ergänzen dazu eine neue Vektorkomponente:

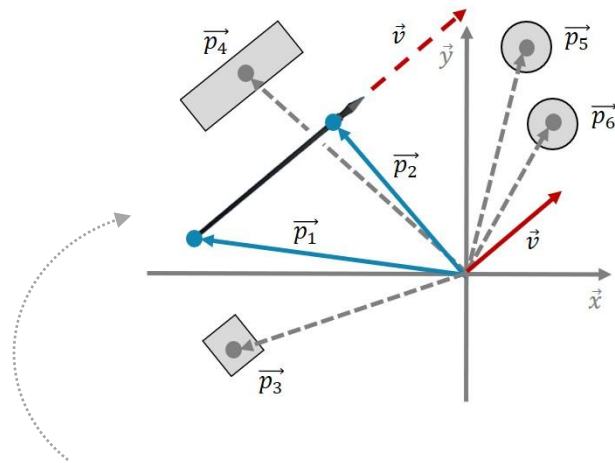
$$\vec{v} = \begin{bmatrix} v_x \\ v_y \end{bmatrix} \Rightarrow \vec{v} = \begin{bmatrix} v_x \\ v_y \\ v_w \end{bmatrix}$$

Entsprechend sieht es bei den 3D-Vektoren aus:

$$\vec{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \Rightarrow \vec{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ v_w \end{bmatrix}$$

Damit können wir eine Transformation (z.B. die Rotation) mit der Translation zu **einer Berechnungsvorschrift** vereinen:

$$\vec{v'} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \Rightarrow \vec{v'} = \begin{bmatrix} x_1 & y_1 & z_1 & t_x \\ x_2 & y_2 & z_2 & t_y \\ x_3 & y_3 & z_3 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ v_w \end{bmatrix}$$



Für unser Beispiel erweitern wir einfach Positionen um 1 und Richtungen um 0

$$\vec{p_1} = \begin{bmatrix} v_x \\ v_y \\ 1 \end{bmatrix} \quad \vec{v} = \begin{bmatrix} v_x \\ v_y \\ 0 \end{bmatrix}$$



Translation mit homogenen Koordinaten

Eine reine Translation im 3D-Raum lässt sich über die homogenen Koordinaten wie folgt umsetzen:

$$M_{\text{translate}} = \begin{bmatrix} I_3 & \vec{t} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Überzeugen wir uns anhand eines 2D-Beispiels mit einem **Positionsvektor** $\vec{p}_1$:

$$M_{\text{translate}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \vec{p}_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot p_x + 0 \cdot p_y + 1 \cdot 1 \\ 0 \cdot p_x + 1 \cdot p_y + 0 \cdot 1 \\ 0 \cdot p_x + 0 \cdot p_y + 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} p_x + 1 \\ p_y \\ 1 \end{bmatrix}$$

Jetzt schauen wir, was bei einem **Richtungsvektor** $\vec{v}$ passiert:

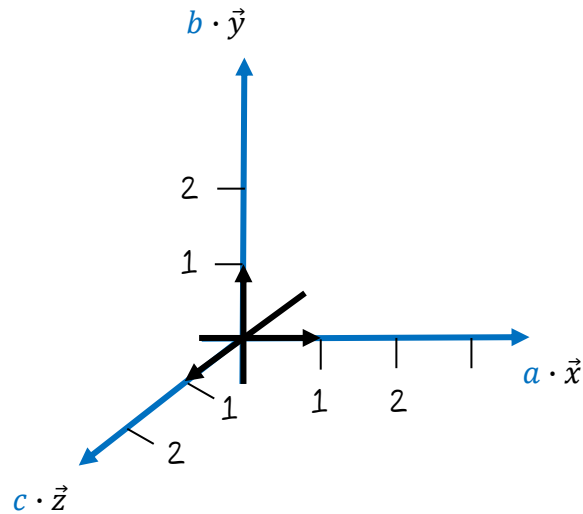
$$M_{\text{translate}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \vec{v} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot v_x + 0 \cdot v_y + 1 \cdot 0 \\ 0 \cdot v_x + 1 \cdot v_y + 0 \cdot 1 \\ 0 \cdot v_x + 0 \cdot v_y + 1 \cdot 0 \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ 0 \end{bmatrix}$$

Es funktioniert! Die Translation wurde für den Richtungsvektor ignoriert. Jetzt erkennen wir auch den Vorteil, Positions- und Richtungsvektoren zu unterscheiden und die homogenen Koordinaten zu verwenden.

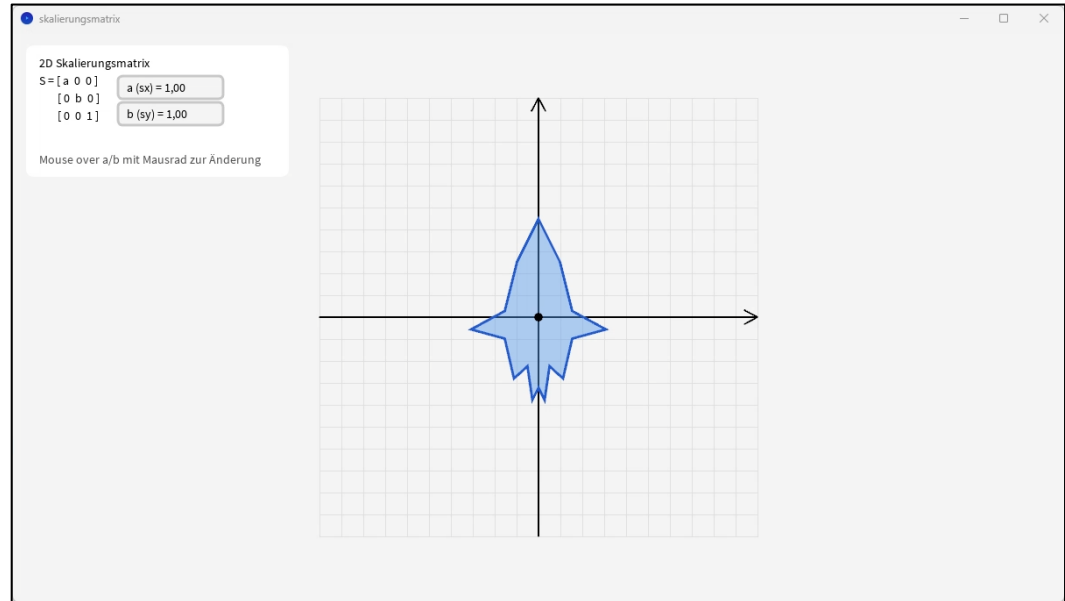
Skalierung mit homogenen Koordinaten

Eine **Skalierung** der jeweiligen Achsen ermöglichen wir jetzt durch folgende Matrix:

$$M_{scale} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Beispiel mit 2D-Skalierungsmatrix

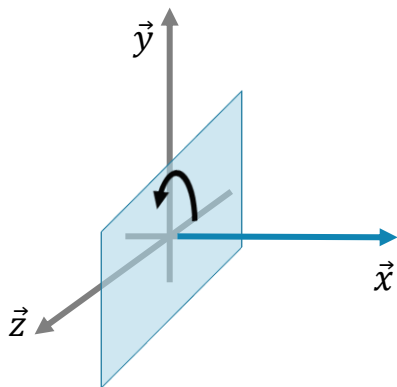


Rotationen mit homogenen Koordinaten

Bei einer [Rotation im 3D-Raum](#) müssen wir uns für eine Rotationsachse entscheiden:

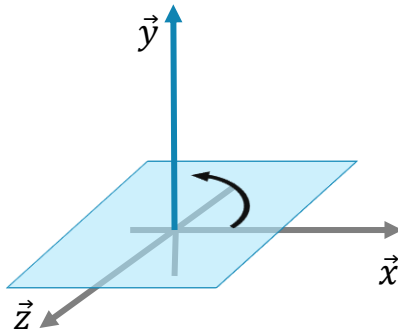
Rotation um x -Achse

$$M_{rotateX} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



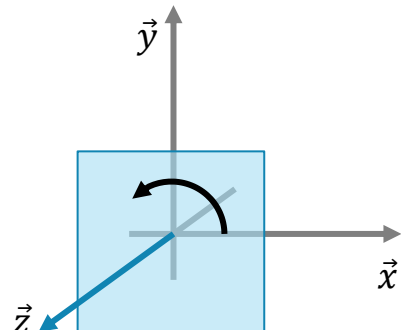
Rotation um y -Achse

$$M_{rotateY} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Rotation um z -Achse

$$M_{rotateZ} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Transformationen beliebig kombinieren

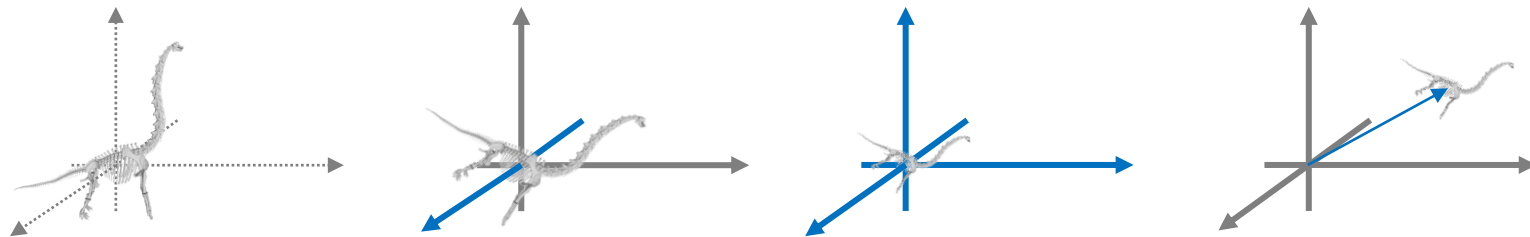
Eine unserer Hauptaufgaben ist die **Kombination sehr unterschiedlicher Transformationen**:

$$\vec{v}' = CBA \cdot \vec{v}$$

Die Reihenfolge der Operationen ist dabei klar, zuerst wird die Transformation A , dann B und schließlich C angewendet:

$$\vec{v}' = C(B(A \cdot \vec{v}))$$

Wir könnten unsere vielen 2D-Positionen des Dinos bspw. rotieren, skalieren und verschieben:



Wenn wir mehrere Vektoren durch diese Transformationskette schicken wollen, lohnt es sich die Matrizen zu einer zusammenzufassen und im Anschluss alle Vektoren mit M zu multiplizieren:

$$M = CBA \quad \Rightarrow \quad \vec{v}' = (CBA) \cdot \vec{v} = M \cdot \vec{v}$$



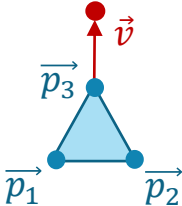
Aber geht das wirklich **schneller**?

Beispiel: Bewegendes Objekt

Konstruieren wir uns als Beispiel ein konkretes Objekt (besteht aus den Punkten $\vec{p}_1$, $\vec{p}_2$ und $\vec{p}_3$), das in eine bestimmte Richtung $\vec{v}$ unterwegs ist. Auf diese Weise wiederholen wir gleich nochmal die Darstellung mit homogenen Koordinaten.

Schauen wir uns zunächst die kodierten Daten an:

$$V = [\vec{p}_1 \quad \vec{p}_2 \quad \vec{p}_3 \quad \vec{v}]$$

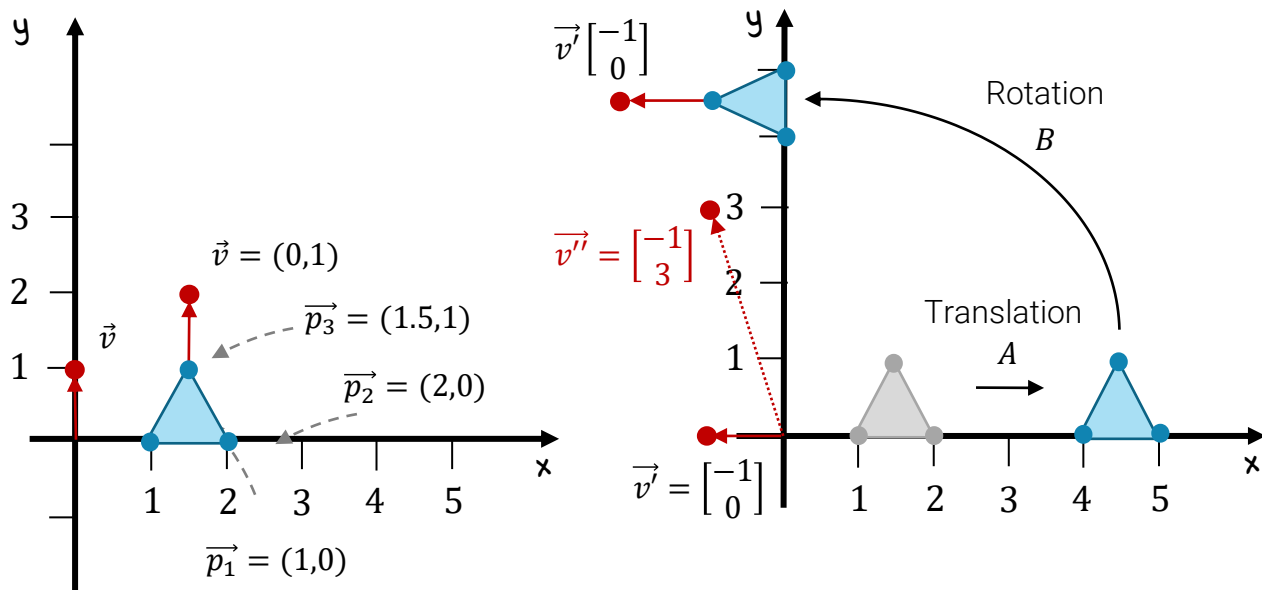
$$V = \begin{bmatrix} 1 & 2 & 1.5 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$


Gegeben seien folgende Transformationen:

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

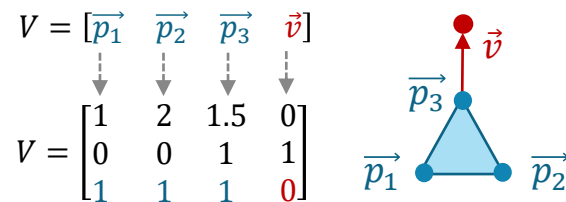
Jetzt transformieren alle homogenen 2D-Vektoren in V mit $V' = (BA) \cdot V$:



Performancevergleich – Vorbereitung

Zunächst erzeugen wir uns ein kleines Dreieck aus drei Positionsvektoren (erste drei Spalten in V) in homogenen Koordinaten und einen Richtungsvektor (vierte Spalte):

```
SimpleMatrix V = new SimpleMatrix(new double[][]  
    {{1,2,1.5,0},  
     {0,0,1,1},  
     {1,1,1,0}});
```



Der Richtungsvektor gibt dem Dreieck eine Bewegungsrichtung nach oben. Weiterhin formulieren wir uns eine Matrix A , die alle Positionsvektoren nach rechts verschiebt, und eine Matrix B , die alle Vektoren um 90° links herum rotiert:

```
SimpleMatrix A = new SimpleMatrix(new double[][]  
    {{1,0,3},  
     {0,1,0},  
     {0,0,1}});  
  
double w = Math.PI/2;  
  
SimpleMatrix B = new SimpleMatrix(new double[][]  
    {{Math.cos(w), -Math.sin(w), 0},  
     {Math.sin(w), Math.cos(w), 0},  
     {0,0,1}});
```

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Performancevergleich – Ergebnis

Wenn wir jetzt die beiden Transformationen für die Objektvektoren V über $B(AV)$ berechnen:

```
long startZeit = System.currentTimeMillis();
for (int i=0; i<100000; i++)
    B.mult(A.mult(V));
long stopZeit = System.currentTimeMillis();
System.out.println("Transformation B(A*V) (100000 Mal "+(stopZeit-startZeit)+" ms)");
System.out.println(B.mult(A.mult(V)));
```

$B(AV)$

Für den direkten Vergleich berechnen wir im Anschluss:

```
startZeit = System.currentTimeMillis();
SimpleMatrix C = B.mult(A);
for (int i=0; i<100000; i++)
    C.mult(V);
stopZeit = System.currentTimeMillis();
System.out.println("Transformation (B*A)V (100000 Mal "+(stopZeit-startZeit)+" ms)");
System.out.println(C.mult(V));
```

$(BA)V$

Als Ergebnis erhalten wir:

```
Transformation B(A*V) (100000 Mal 42 ms)
Transformation (B*A)V (100000 Mal 20 ms)
```



Sind die Ergebnisse überhaupt **identisch**?

Vergleich der Ergebnisse – Konsequenz

Am Ende wollen wir noch die Ergebnisse vergleichen:

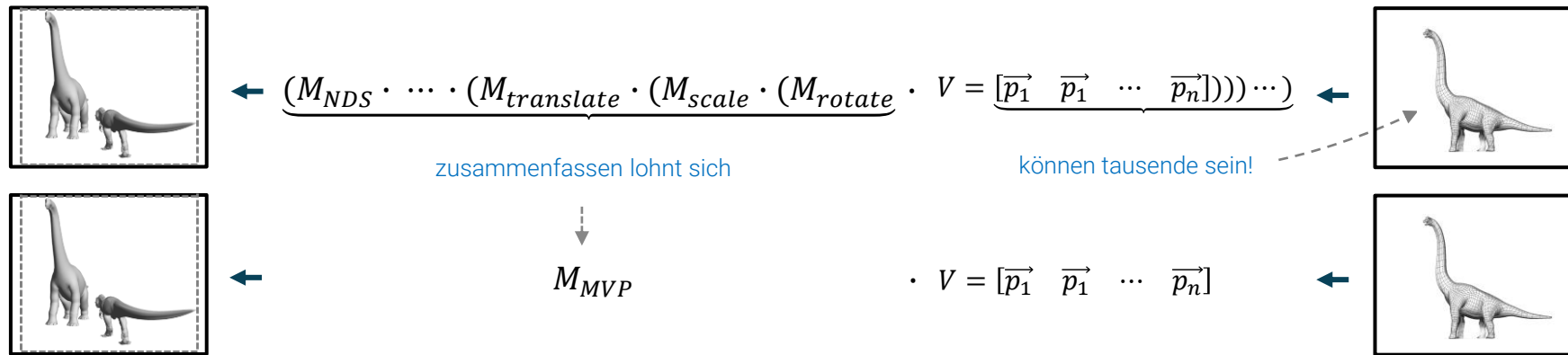
```
if (B.mult(A.mult(V)).isIdentical(C.mult(V), 0.0001))  
    System.out.println("B(A*V) und (B*A)V liefern das gleiche Ergebnis.\n");
```

Als Ergebnis erhalten wir:

B(A*V) und (B*A)V liefern das gleiche Ergebnis.



Was das für uns bedeutet ([Transformationen immer zusammenzufassen](#)):





An diesem Beispiel wollen wir noch die
Inversenbildung kurz untersuchen

$$\begin{aligned}
 (A+B)^T &= A^T + B^T \\
 (c \cdot A)^T &= c \cdot A^T \\
 (A \cdot B)^T &= B^T \cdot A^T \\
 (A^{-1})^T &= (A^T)^{-1}
 \end{aligned}$$

$$\begin{aligned}
 MV &= (B \cdot A)V \\
 M^{-1} &= (B \cdot A)^{-1} = (A^{-1} \cdot B^{-1}) = (A^T \cdot B^T) = M^T \\
 M^{-1} MV &= V
 \end{aligned}$$

Inverse Transformation dieses Beispiels

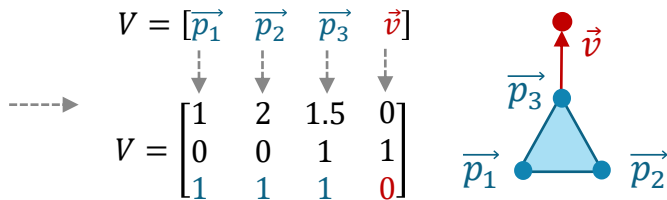
Wenn wir die gleiche Datenbasis wie im vorhergehenden Beispiel nehmen und als Rücktransformation $(BA)^{-1} = A^{-1}B^{-1}$ einsetzen, dann müssen wir nur das Resultat folgender Zeile betrachten:

```
System.out.println((A.invert().mult(B.invert()).mult(C.mult(V))));
```

Die Ausgabe entspricht unserer Datenbasis zu Beginn:

```
Type = DDRM , rows = 3 , cols = 4  
1,000  2,000  1,500  0,000  
0,000  0,000  1,000  1,000  
1,000  1,000  1,000  0,000
```

```
SimpleMatrix V = new SimpleMatrix(new double[][]  
    {{1,2,1.5,0},  
     {0,0,1,1},  
     {1,1,1,0}});
```

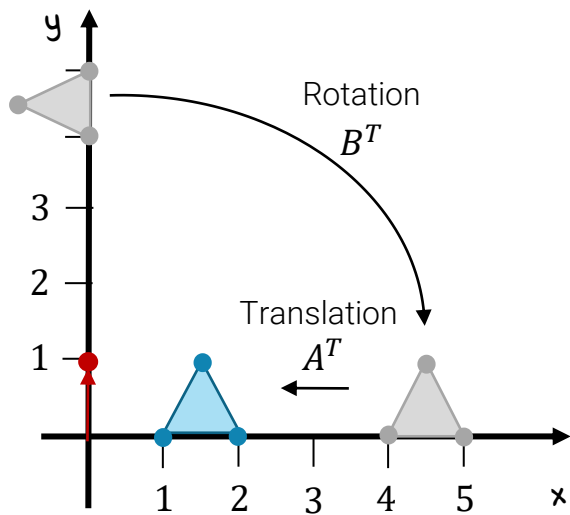


Jetzt haben wir für unser Beispiel auch die inverse Transformation konkret angewendet. Im Übrigen steckt in der Rücktransformation $(BA)^{-1} = A^{-1}B^{-1}$ unser intuitives Verständnis zur Verkettung von verschiedenen Transformationen: Wenn wir uns durch A verschieben und danach um den Mittelpunkt mit B rotieren, müssen wir uns bei der inversen Bewegung erst wieder zurückdrehen mit B^{-1} und dann zurück verschieben um A^{-1} .

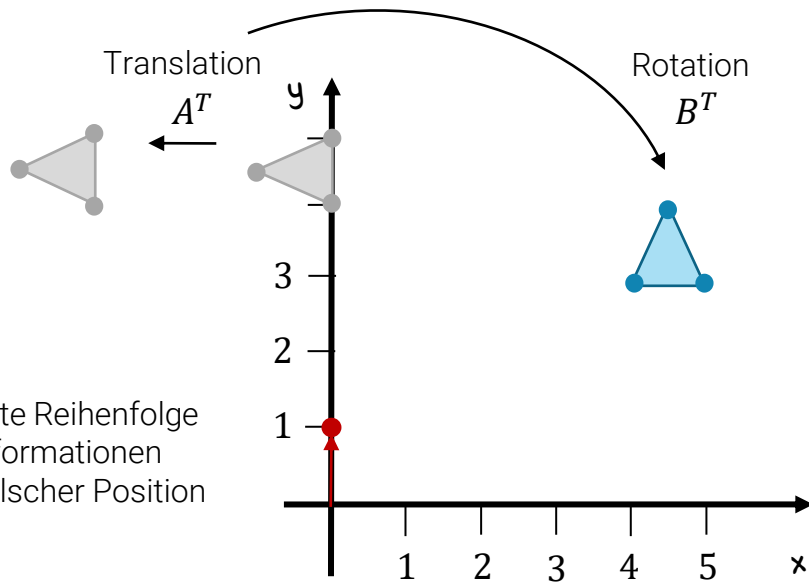
Inverse Transformation dieses Beispiels

Die korrekte Reihenfolge bei der inversen Transformation führt auch wieder zur ursprünglichen Situation. Sollten wir die Transformationen vertauschen, kann das wie in diesem Fall zu einem falschen Ergebnis führen. Wir wissen ja, dass die **Matrixmultiplikation nicht kommutativ** ist!

Inverse Transformation



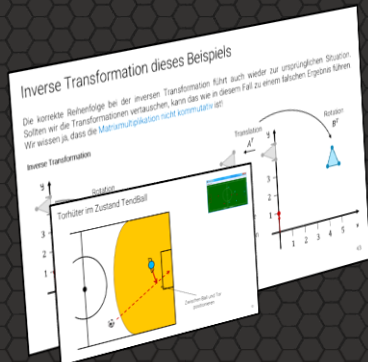
Vertauschte Reihenfolge
der Transformationen
führt zu falscher Position



Transformationen als Vollzeitjob

Key Takeaways

- Koordinatentransformationen können Entscheidungen vereinfachen
- Die Wahl des Bezugssystems ist entscheidend
- Rotationen sind orthogonale Transformationen
- Translationen benötigen homogene Koordinaten
- Matrixmultiplikation ist nicht kommutativ aber assoziativ
- Transformationen lassen sich effizient kombinieren
- Inverse Transformationen kehren die Reihenfolge um
- Punkte und Richtungen müssen unterschiedlich behandelt werden
- Performance profitiert von Vorab-Kombination



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Geben Sie die 2D-Rotationsmatrix für einen beliebigen Winkel θ an und zeigen Sie explizit, dass $R^T R = I$.
- (2) Warum ist die homogene Matrizendarstellung von Rotation, Skalierung und Translation der Standarddarstellung vorzuziehen?
- (3) Erklären Sie anhand eines Beispiels, warum Richtungsvektoren nicht translatiert werden dürfen.
- (4) Warum kehrt sich bei der Inversenbildung die Reihenfolge der Transformationen um?

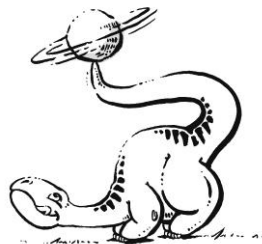


Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Rotieren Sie den Vektor $(2,1,5)$ in 135° um die z -Achse. Verwenden Sie dabei die entsprechende Rotationsmatrix und nutzen Sie dafür EJML (Tipp wandeln Sie zuerst die Winkelangabe z.B. mit `Math.toRadians` in ein Bogenmaß um).

Aufgabe 2) [Java] Überlegen Sie sich ein einfaches Setup mit Spielerpositionen und dem Ball. Ein Spieler hat den Ball, es gibt Gegner und Mitspieler. Implementieren Sie eine Funktion, die entscheidet, ob ein Pass zu einem der Mitspieler sicher ist oder nicht.





Vielen Dank für die Aufmerksamkeit